

END SEMESTER ASSESSMENT (ESA) - Jan - May 2024**UE18CS351 - Compiler Design****Total Marks : 100.0**

1.a. i) Define Interpreter. Explain in brief how it is different from a Compiler.
ii) Explain in brief any two compiler types. (4.0 Marks)

1.b. Why is buffering used in lexical analysis? What are the commonly used buffering methods? What is the advantage of having sentinels at the end of each buffer halves in buffer pairs? (4.0 Marks)

1.c. With a diagram explain in brief the design of a lexical analyzer. (4.0 Marks)

1.d. What are Lexical Errors? Write down the sequence of lexemes for the following code segment

```
while(i <= 100) { printf("%d",i); i++; }
```

(4.0 Marks)

1.e. i) What is Lex tool? Mention the sections of Lex specification program.

ii) Consider the following lexical specification:

%%

a*b printf("Jai ");

ca printf("Sita ");

a*ca* printf("Ram ");

Given the following input string:

abcaacacaaa

What does the lexer print?

(4.0 Marks)

2.a. Explain with an example any two Error-Recovery Strategies.

(4.0 Marks)

2.b. Consider the following grammar:

$S \rightarrow +SS \mid *SS \mid a$

1. Calculate FIRST and FOLLOW

2. Construct predictive parsing table

3. Show how to parse the input *+aa*

(6.0 Marks)

2.c. Consider the following grammar:

S → **AaAb**

S → **BbBa**

A → ϵ

B → ϵ

1. Construct LR(0) automata

2. Construct SLR parsing table to check whether the given grammar is SLR or not.

(6.0 Marks)

2.d. i) Explain in brief Canonical LR(1) Items.

ii) Why is LR(1)/CLR(1) parser so powerful?

(4.0 Marks)

3.a. Explain the following in detail

i) Synthesized Attribute and Inherited Attribute

ii) L-attributed SDT and S-attributed SDT

(8.0 Marks)

3.b. Consider the following Context Free Grammar for variable declaration in C language:

$D \rightarrow TL$

$T \rightarrow \text{int} \mid \text{float} \mid \text{char} \mid \text{double}$

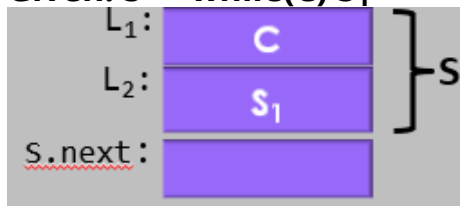
$L \rightarrow L_1, \text{id} \mid \text{id}$

Write an Syntax-Directed Definition (SDD) to add/update type (i.e., int/float/char/double) information of the variables in declaration statement to the symbol table during parsing. (4.0 Marks)

3.c. Write syntax directed definition (SDD) to generate intermediate code for while loop.

Given: $S \rightarrow \text{while}(C) S_1$

(4.0 Marks)



3.d. Consider, SDT for if-else statement

```
S -> if ({ L1=new(); L2=new(); C.false=L2; C.true=L1;} C)  
    {S1.next=S.next;} S1 else {S2.next=S.next;} S2  
    {S.code=C.code || label || L1 || S1.code || label || L2 || S2.code}
```

Let us turn this SDT(L-attributed) into an SDT that can operate with an LR parse (i.e., suitable during bottom-up parsing). (4.0 Marks)

4.a. Write short notes on:

- i) Quadruple
- ii) Static Single Assignment (SSA)

(5.0 Marks)

4.b. Construct control-flow graph (CFG) for the following three-address code

```
x = 0
y = 0
L1: ifFalse x < 10 goto L2
    t1 = y + x;
    y = t1
    t2 = x + 1;
    x = t2
    goto L1
L2: print y
```

(5.0 Marks)

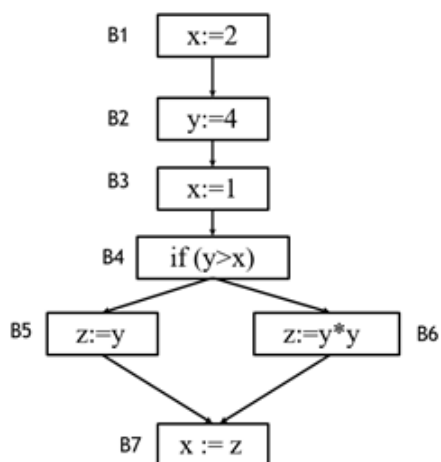
4.c. Optimize the code below by applying the following code transformation techniques: Constant Folding, Constant Propagation and dead code elimination.

```
int a = 30;
int b = 9 - (a / 5);
int c;
c = b * 4;

if (c > 10) {
    c = c - 10;
}
return c * (60 / a);
```

(5.0 Marks)

4.d. Calculate use[B], def[B] and then find IN[B], OUT[B] by applying *Live-variable Analysis Algorithm* on the following flow graph.



5.a. Explain in brief any four issues in the design of a code generator. (4.0 Marks)

5.b. Explain the following with a neat diagram

i) Activation record

ii) Activation tree

(6.0 Marks)

5.c. Explain in brief the different ways in which a procedure can access nonlocal data on a runtime Stack. (4.0 Marks)

5.d. Generate target code sequence for the following basic block three address statements with register and address descriptors.

1. $x := y - z$

2. $z := x * x$

3. $y := z$

4. $x := y - z$

Note 1: Assuming only two registers (R1 and R2) are available and all the variables are live on exit from the basic block. No temporary variables are used.

Note 2: Assuming that all arithmetic operations take their operands from registers. (6.0 Marks)